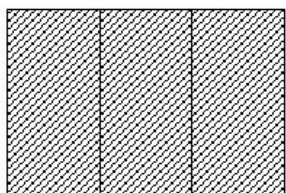


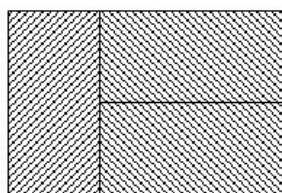


【例2.10】 骨牌铺方格（HDU2046）。在 $2 \times n$ 的一个长方形方格中，用 1×2 的骨牌铺满方格。输入 n ，输出铺放方案的总数。

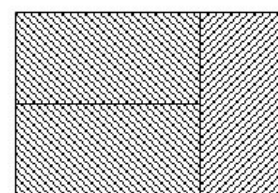
例如 $n=3$ 时，为 2×3 方格，骨牌的铺放方案有如图所示的3种。



(a) 方案1



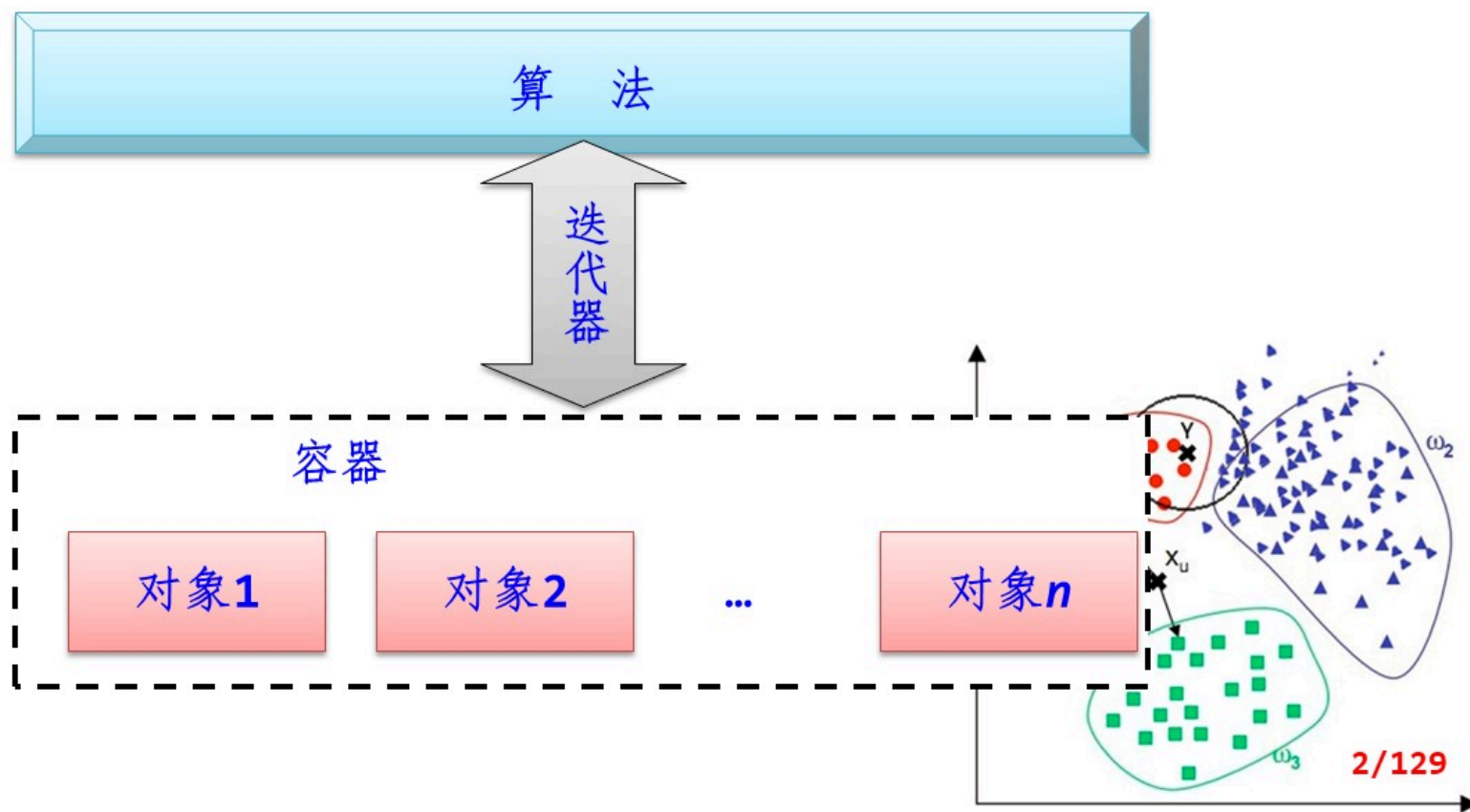
(b) 方案2



(c) 方案3

– 知识回顾与问题导入

- STL工具的应用可以简化算法的细节，提高算法设计的效率。
- STL是有容器、算法和迭代器三类组成的。



– 知识回顾与问题导入

- STL定义好的容器

数据结构容器	说 明	头文件
vector	向量	<vector>
string	字符串	<string>
deque	双端队列	<deque>
list	链表	<list>
stack	栈	<stack>
queue	队列	<queue>
priority_queue	优先队列	<queue>
set/multiset	集合/多重集合	<set>
map/multimap	映射/多重映射	<map>
unordered_set	哈希集合	<unordered_set>
unordered_map	哈希映射	unordered_map>

3/129

单选题 1分

数组中的第 k 个最大元素（**LeetCode215**★★）。设计一个算法求无序数组`nums`中第 k （ $1 \leq k \leq$ 数组的长度）大的整数，注意不是第 k 个不同的元素，而是数组排序后的第 k 个最大的元素。

例如，`nums={3, 2, 1, 5, 6, 4}`， $k=2$ ，`nums`递减排序后是`{6, 5, 4, 3, 2, 1}`，答案为5，若 $k=5$ ，答案为2。

要求设计如下成员函数：使用了什么容器对象？

```
int findKthLargest(vector<int>& nums, int k) { }
```

A string

B vector

C list

D queue

4/91

第2章—递归算法设计技术

提纲 CONTENTS

2.1 递归概述

2.2 递归算法设计方法

2.3 直接插入排序

2.4 0/1背包问题

2.5 求表达式值

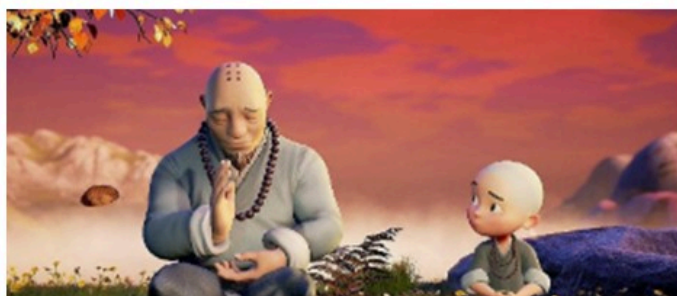
2.6 计算递推式

5/91

2.1 递归概述

2.1.1 什么是递归

老和尚给小和尚讲故事：“从前有座山，山上有座庙，庙里有个老和尚和一个小和尚，一天，老和尚给小和尚讲一个故事，故事是‘从前有座山，山上有座庙，庙里有个老和尚和一个小和尚，一天，老和尚给小和尚讲一个故事，故事是…’” ⇒ **递归**。



6/91

- **递归 (recursion)** 是一个过程或函数在其定义或说明中直接或间接调用自身的一种方法（摘自《百度百科》词条）。
- **递归** 既是一种奇妙的现象，又是一种思考问题的方法，通过递归可简化问题的定义和求解过程。

单选题 1分

汉诺塔问题，从A塔，移动4个盘子到B塔，C塔作为中转塔。每次只能移动1个盘子，每个塔上只能大盘子在下小盘子在上，共需要几步。

- ☐ A 3
- ☐ B 7
- ☐ C 14
- ☒ D 15

8/91

汉诺塔问题，从A塔，移动4个盘子到B塔，C塔作为中转塔。每次只能移动1个盘子，每个塔上只能大盘子在下小盘子在上。

请写出移动过程。

【例2.1】 设计求 $n!$ （ n 为正整数）的递归算法。



```
int fun(int n) {  
    if (n==1)                //语句1  
        return 1;           //语句2  
    else                      //语句3  
        return fun(n-1)*n;   //语句4  
}
```

一般来说，能够用递归解决的问题应该满足以下3个条件：

- ① 需要解决的问题可以转化为一个或多个子问题来求解，而这些子问题的求解方法与原问题完全相同，只是问题规模上不同。通过这样的转化过程使问题得到简化。
- ② 递归调用的次数必须是有限的。
- ③ 必定有一个明确的结束递归的条件，否则递归将会无止境地进行下去，直到耗尽系统资源。也就是说必须要有某个终止递归的条件。

2.1.2 何时使用递归

1. 定义是递归的

- 有许多数学公式、数列和概念的定义是递归的，例如，求 $n!$ 和斐波那契（**Fibonacci**）数列等。
- 这些问题的求解过程可以将其递归定义直接转化为对应的递归算法。

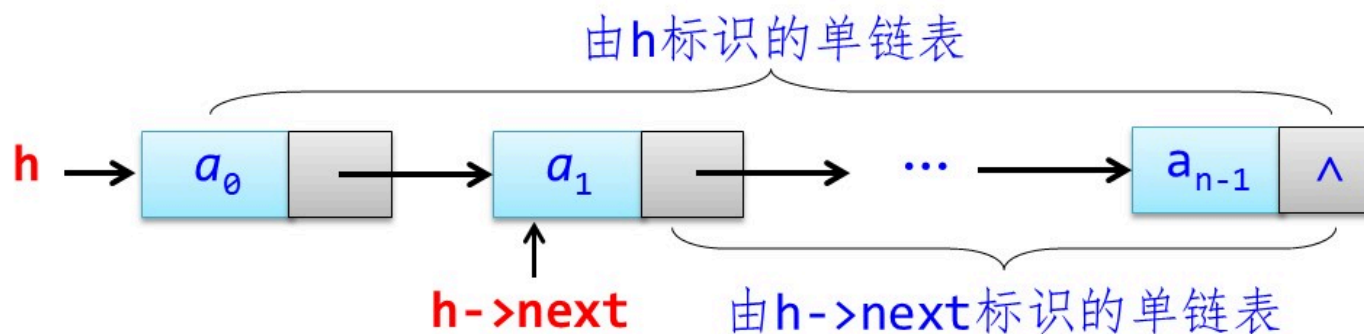
2. 数据结构是递归的

- 算法是用于数据处理的，有些存储数据的数据结构是递归的。
- 对于递归数据结构，采用递归的方法设计算法既方便又有效。

单链表就是一种递归数据结构，其结点类型声明如下：

```
struct ListNode {                                //单链表结点类型
    int val;
    ListNode *next;

    ListNode():val(0), next(NULL) {}             //构造函数
    ListNode(int x):val(x),next(NULL) {}
    ListNode(int x, ListNode *next):val(x),next(next) {}
};
```

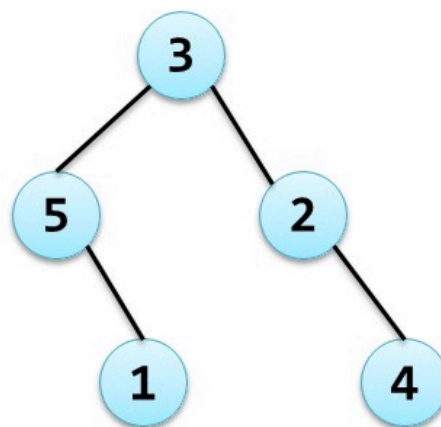


12/91

求一个不带头结点的单链表h的所有val域之和的递归算法如下：

```
int Sum(ListNode *h) { //求不带头结点单链表h中所有结点值之和
    if (h==NULL)
        return 0;
    else
        return h->val+Sum(h->next);
}
```

【例2.2】分析整数二叉树的二叉链存储结构的递归性，设计求非空二叉链 r 中所有结点值之和的递归算法。



答案为**15**

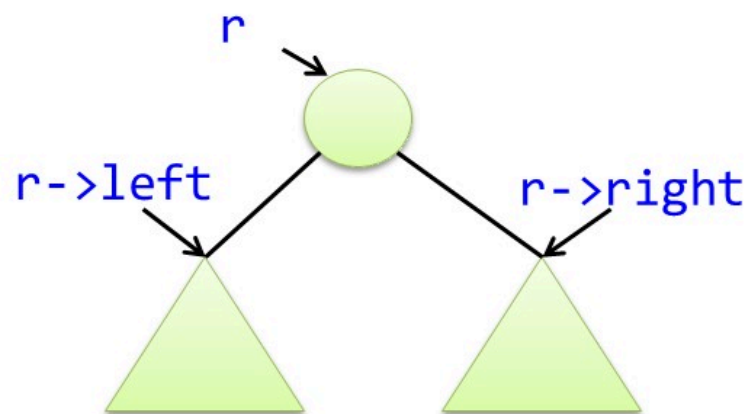
解

假设整数二叉树采用二叉链存储结构，所有结点值为 **int** 类型。二叉链的结点类型声明如下：

```
struct TreeNode {                                //二叉链结点类型
    int val;                                       //结点值（为int类型）
    TreeNode *left;                             //左孩子结点指针
    TreeNode *right;                            //右孩子结点指针

    TreeNode() : val(0), left(NULL), right(NULL) {}
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
    TreeNode(int x, TreeNode *left, TreeNode *right):
        val(x), left(left), right(right) {}
};
```

15/91



- **r**指向根结点，用于标识整棵树。
- **r->left**和**r->right**分别指向左、右孩子结点，用于标识左、右子树，而左、右子树本身也都是二叉树。
- 是一种**递归数据结构**。

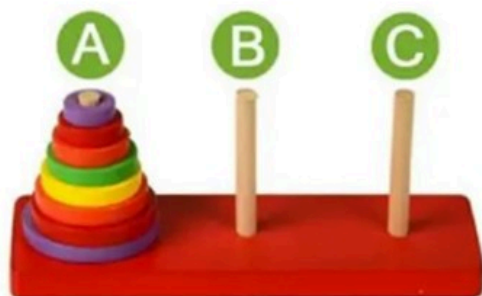
递归算法

```
int Sumbt(TreeNode *r) {    //求二叉树bt中所有结点值之和
    if (r==NULL)            //空树返回0
        return 0;
    else                    //非空树返回左、右子树结点值之和加上根结点值
        return Sumbt(r->left)+Sumbt(r->right)+r->val;
}
```

3.问题的求解方法是递归的

例如汉诺塔问题。

- 3个分别命名为 x 、 y 和 z 的塔座，在塔座 x 上有 n 个直径各不相同、从小到大依次编号为 $1\sim n$ 的盘片，现要求将 x 塔座上的 n 个盘片移到塔座 z 上并按同样顺序叠放。
- 盘片移动时必须遵守以下规则：每次只能移动一个盘片；盘片可以插在 x 、 y 或 z 中任一塔座；任何时候都不能将一个较大的盘片放在较小的盘片上。



18/91

主观题 10分

- 3个分别命名为 x 、 y 和 z 的塔座，在塔座 x 上有 n 个直径各不相同、从小到大依次编号为 $1 \sim n$ 的盘片，现要求将 x 塔座上的 n 个盘片移到塔座 z 上并仍按同样顺序叠放。
- 盘片移动时必须遵守以下规则：每次只能移动一个盘片；盘片可以插在 x 、 y 或 z 中任一塔座；任何时候都不能将一个较大的盘片放在较小的盘片上

`Hanoi(n, x, y, z)`



```
Hanoi(n-1, x, z, y);  
move(n, x, z): 将第n个盘片从x移到z;  
Hanoi(n-1, y, x, z)
```

21/91

解

$\text{Hanoi}(n, x, y, z)$



```
Hanoi( $n-1$ ,  $x$ ,  $z$ ,  $y$ );  
move( $n$ ,  $x$ ,  $z$ ): 将第 $n$ 个盘片从 $x$ 移到 $z$ ;  
Hanoi( $n-1$ ,  $y$ ,  $x$ ,  $z$ )
```

20/91

```
void Hanoi(int n,char x,char y,char z) {  
    if (n==1)  
        printf("将盘片%d从%c搬到%c\n",n,x,z);  
    else {  
        Hanoi(n-1,x,z,y);  
        printf("将盘片%d从%c搬到%c\n",n,x,z);  
        Hanoi(n-1,y,x,z);  
    }  
}
```

2.1.3 递归模型

- 递归思想的核心可以用递归模型表示，递归模型是递归算法的抽象，它反映一个递归问题的递归结构。
- 一般地，一个递归模型是由递归出口和递归体两部分组成，递归出口确定递归到何时结束，即指出明确的递归结束条件。

```
int fun(int n) {  
    if (n==1)                //语句1  
        return 1;           //语句2  
    else                      //语句3  
        return fun(n-1)*n;   //语句4  
}
```



抽象

递归出口 

$f(n)=1$

$n=1$

递归体 

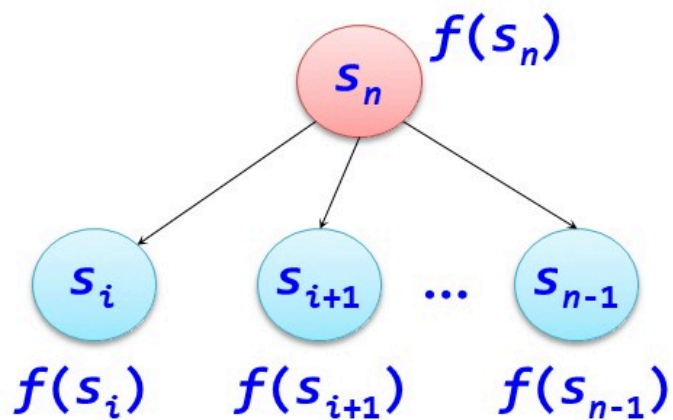
$f(n)=n*f(n-1)$

$n>1$

递归模型:

$$f(s_1)=m_1$$

$$f(s_n)=g(f(s_i), f(s_{i+1}), \dots, f(s_{n-1}), c_j, c_{j+1}, \dots, c_m)$$



2.1.4 递归算法的执行过程

(1) 递归执行是通过系统栈实现的。

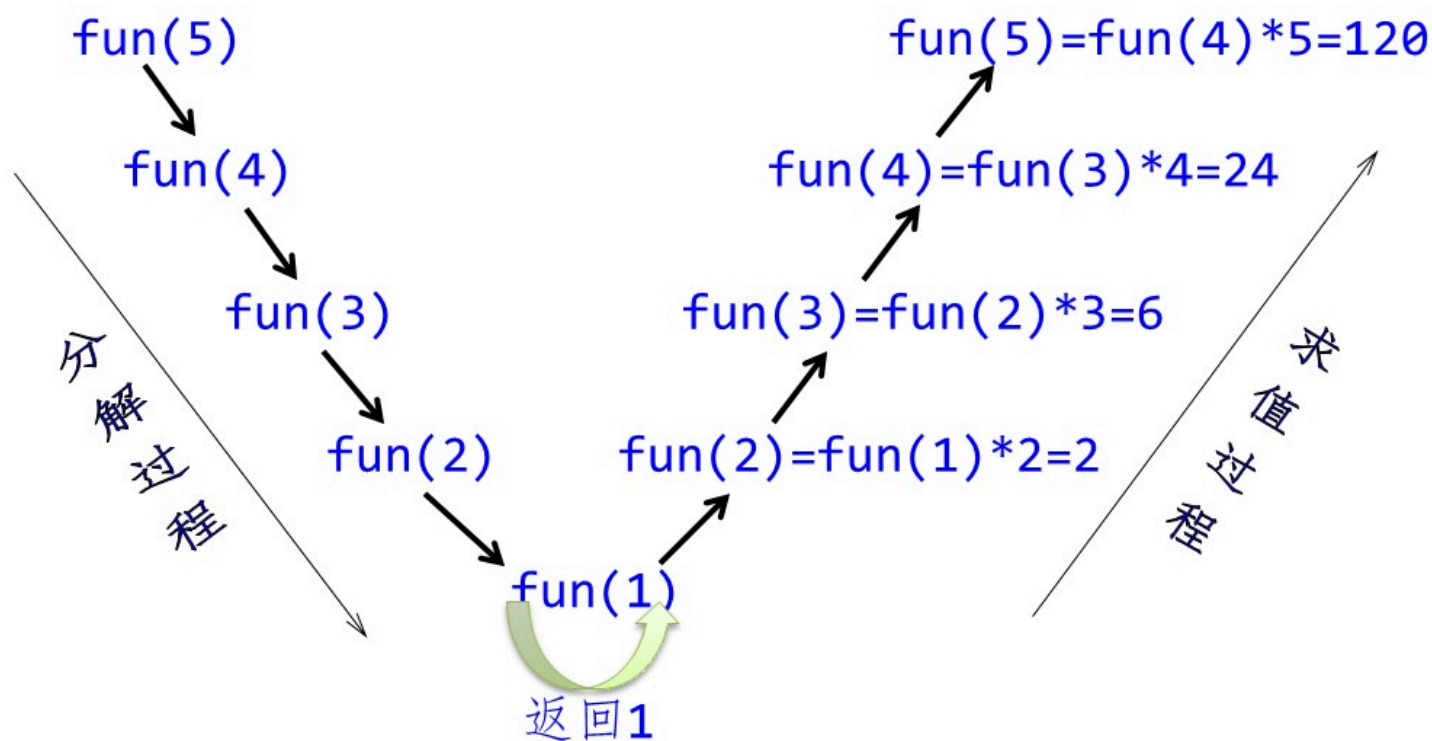
(2) 每递归调用一次就需将参数、局部变量和返回地址等作为一个栈元素进栈一次，最多的进栈元素个数称为递归深度，当问题规模 n 越大，递归深度越深。

(3) 每当遇到递归出口或本次递归调用执行完毕，需退栈一次，并恢复参数值等，当全部执行完毕时，栈应为空。

```

int fun(int n) {
    if (n==1)                //语句1
        return 1;           //语句2
    else                      //语句3
        return fun(n-1)*n;   //语句4
}

```



26/91

【例2.3】斐波那契数列定义为：

$\text{Fib}(n)=1$ 当 $n=1$ 时

$\text{Fib}(n)=1$ 当 $n=2$ 时

$\text{Fib}(n)=\text{Fib}(n-1)+\text{Fib}(n-2)$ 当 $n>2$ 时

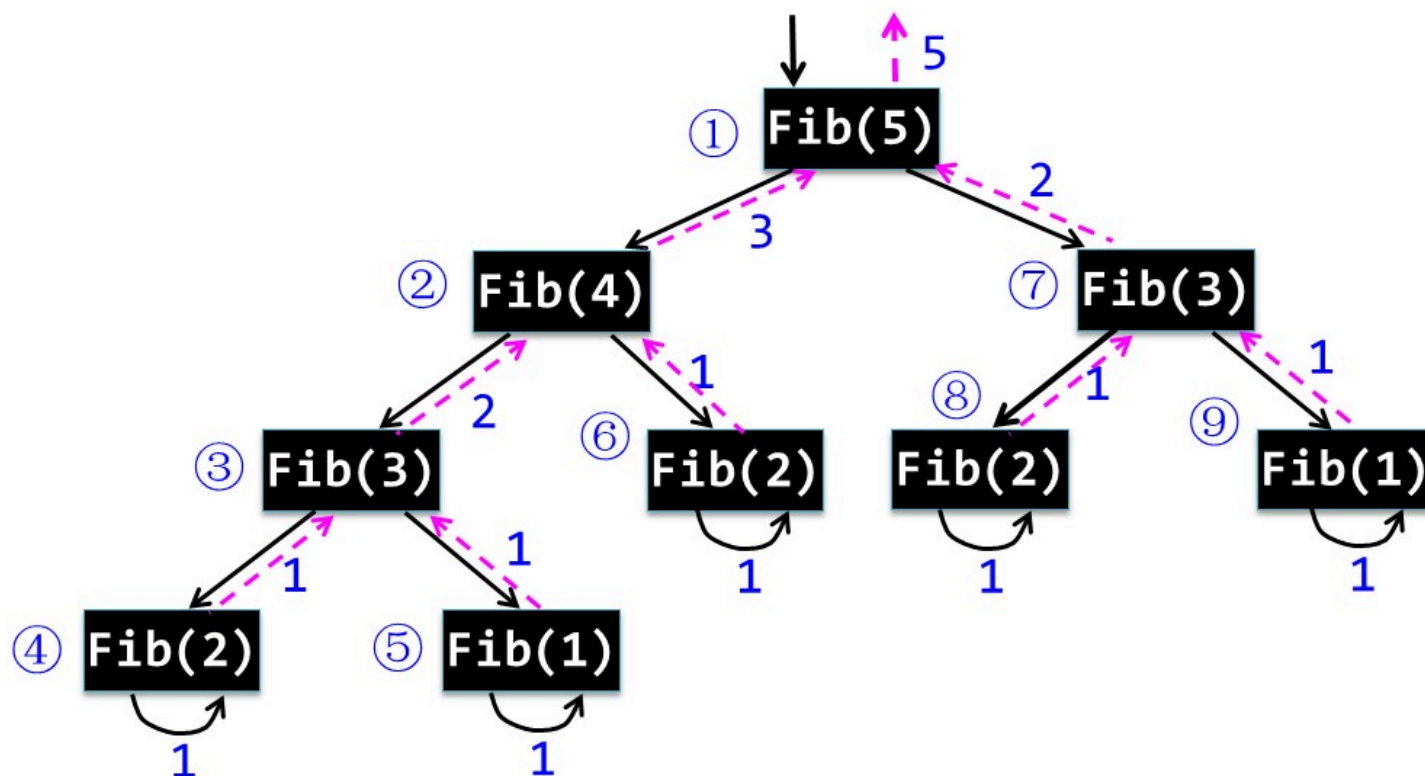
对应的递归算法如下：

```
int Fib(int n) { //递归求Fibonacci数列
    if (n==1 || n==2)
        return 1;
    else
        return Fib(n-1)+Fib(n-2);
}
```

画出求**Fib(5)**的递归树。

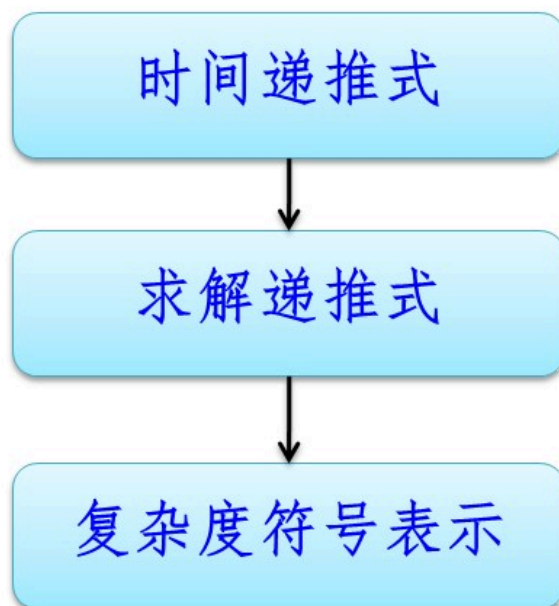
解

求**Fib(5)**的递归树如图所示。图中方框旁的数字表示递归调用的次序，实箭头线表示分解关系，虚箭头线表示求值关系，虚箭头线上的数字表示求值结果。



2.1.4 递归算法的时间和空间分析

1. 递归算法的时间复杂度分析



示例

```
void Hanoi(int n,char x,char y,char z) {  
    if (n==1)  
        printf("将盘片%d从%c搬到%c\n",n,x,z);  
    else {  
        Hanoi(n-1,x,z,y);  
        printf("将盘片%d从%c搬到%c\n",n,x,z);  
        Hanoi(n-1,y,x,z);  
    }  
}
```

设Hanoi(n , x , y , z)的执行时间为 $T(n)$, 则两个问题规模为 $n-1$ 的子问题的执行时间均为 $T(n-1)$, 总时间是累加关系。

$$T(n)=1$$

当 $n=1$ 时

$$T(n)=2T(n-1)+1$$

当 $n>1$ 时

30/91

$$T(n)=1$$

当 $n=1$ 时

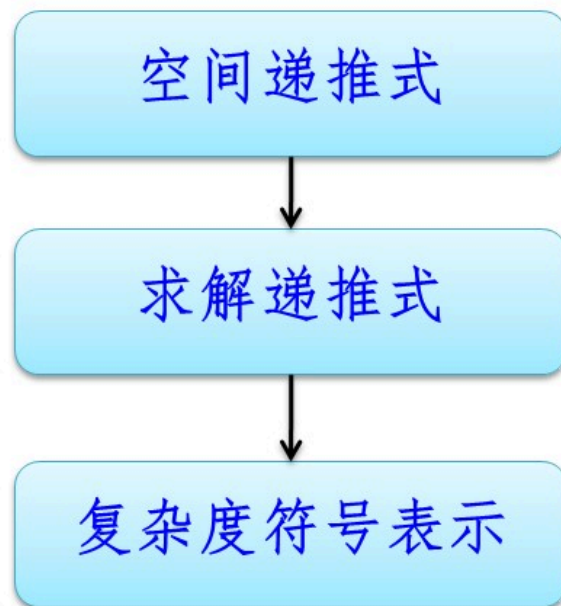
$$T(n)=2T(n-1)+1$$

当 $n>1$ 时



$$\begin{aligned} T(n) &= 2T(n-1)+1 = 2(2T(n-2)+1)+1 \\ &= 2^2T(n-2)+2+1 = 2^2(2T(n-3)+1)+2+1 \\ &= 2^3T(n-3)+2^2+2+1 \\ &= \dots \\ &= 2^{n-1}T(1)+2^{n-2}+\dots+2^2+2+1 \\ &= 2^n-1 \\ &= O(2^n)。 \end{aligned}$$

2. 递归算法的空间复杂度分析



示例

```
void Hanoi(int n,char x,char y,char z) {  
    if (n==1)  
        printf("将盘片%d从%c搬到%c\n",n,x,z);  
    else {  
        Hanoi(n-1,x,z,y);  
        printf("将盘片%d从%c搬到%c\n",n,x,z);  
        Hanoi(n-1,y,x,z);  
    }  
}
```

设Hanoi(n , x , y , z)的临时空间为 $S(n)$, 两个问题规模为 $n-1$ 的子问题的临时空间均为 $S(n-1)$, 总空间是最大值关系, 因为前一个子问题执行完毕其空间被释放, 释放的空间被后一个子问题重复使用。

$$S(n)=1$$

当 $n=1$ 时

$$S(n)=S(n-1)+1$$

当 $n>1$ 时

$$S(n)=1$$

当 $n=1$ 时

$$S(n)=S(n-1)+1$$

当 $n>1$ 时



$$S(n)=S(n-1)+1=(S(n-2)+1)+1$$

$=\cdots$

$$=S(1)+1+\cdots+1$$

$$=1+1+\cdots+1$$

$$=O(n)$$

2.2 递归算法设计方法

2.2.1 递归和数学归纳法

第一数学归纳法原理：若 $\{P(1), P(2), P(3), P(4), \dots\}$ 是命题序列且满足以下两个性质，则所有命题均为真：

- (1) 归纳基础： $P(1)$ 为真。
- (2) 归纳步骤：任何命题均可以从它的前一个命题推导得出。

第二数学归纳法原理：若 $\{P(1), P(2), P(3), P(4), \dots\}$ 是满足以下两个性质的命题序列，则对于其他自然数，该命题序列均为真：

(1) 归纳基础： $P(1)$ 为真。

(2) 归纳步骤：任何命题均可以从它的前面所有命题推导得出。

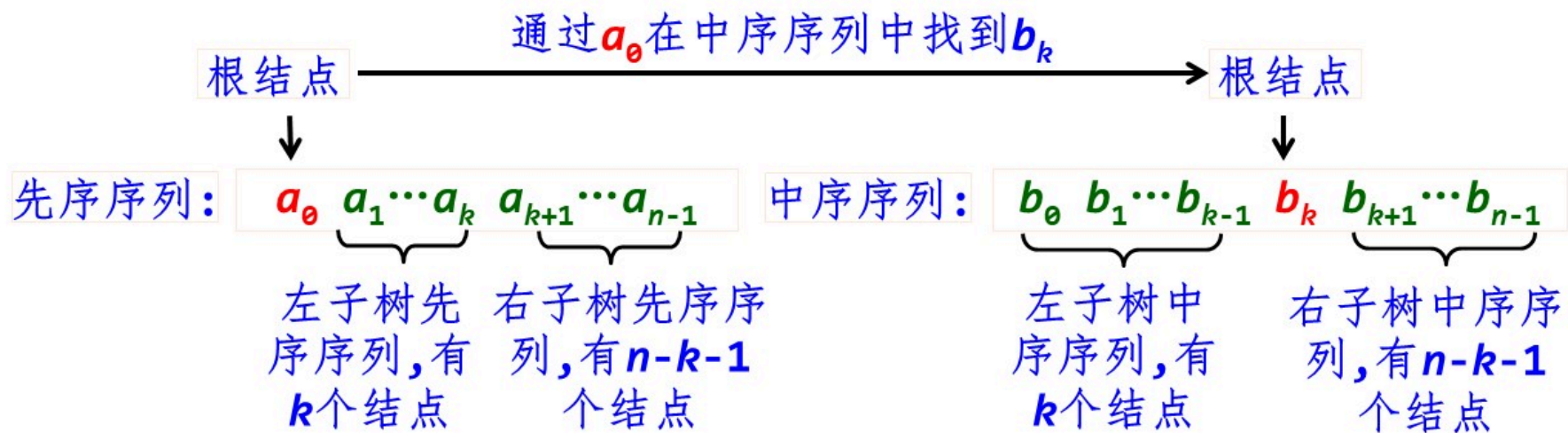
其中归纳步骤的意思是 $P(n)$ 可以从前面所有命题假设 $\{P(1), P(2), P(3), \dots, P(n-1)\}$ 推导得出。

【例2.4】采用第二数学归纳法证明，任何含有 n ($n \geq 0$) 个不同结点值的二叉树，都可由它的中序序列和先序序列唯一地确定。

证明

采用第二数学归纳法。 k 表示二叉树中结点个数。

- (1) $k=0$ 时成立。
- (2) 假设 $k < n$ 时均成立



二叉树的3个部分均可以唯一确定 $\Rightarrow k=n$ 时成立!



- 递归与数学归纳法本质是相同的。
- 数学归纳法是一种证明方法，而递归是一种算法设计或者编程技术，数学归纳法是递归的理论基础，所以实际中往往利用数学归纳法思路来设计递归算法。

2.2.2 递归算法设计的一般步骤

- 递归算法设计的关键是提取求解问题的递归模型。
- 在此基础上转换成对应的递归算法。

提取求解问题的递归模型

- (1) 对原问题 $f(s_n)$ 进行分析，找出合理的小问题 $f(s_{n-1})$ 。
- (2) 假设 $f(s_{n-1})$ 是可解的，在此基础上确定 $f(s_n)$ 的解，即给出 $f(s_n)$ 与 $f(s_{n-1})$ 之间的关系 $\Rightarrow$ 递归体。
- (3) 找到一个特定的可以直接求解的情况 $\Rightarrow$ 递归出口。

【例2.6】 给定一个含 n ($n>2$) 个整数的数组 a ，设计一个递归算法求其中最大元素。

解

设大问题 $f(a, \text{low}, \text{high})$ 求 $a[\text{low}..\text{high}]$ 中的最大元素。

$f(a, \text{low}, \text{high}) = a[\text{low}]$

当 $\text{low}=\text{high}$ 时

$f(a, \text{low}, \text{high}) = \text{MAX}\{\text{lmaxe}, \text{rmaxe}\}$

其他情况

其中 $\text{lmaxe} = f(a, \text{low}, \text{mid})$

$\text{rmaxe} = f(a, \text{mid}+1, \text{high})$

```
int maxe(int a[],int low,int high) {    //递归算法
    if (low==high)
        return a[low];
    else {
        int mid=(low+high)/2;
        int lmaxe=maxe(a,low,mid);
        int rmaxe=maxe(a,mid+1,high);
        return max(lmaxe,rmaxe);
    }
}
```

2.2.3 基于递归数据结构的递归算法设计

- 像单链表、二叉树等都是递归数据结构。
- 递归数据结构中很容易找到其中的基本递归运算，例如二叉树中取一个结点 (**r**) 的左右子树即 **r->left** 和 **r->right** 就是基本递归运算，因为取出的子树也一定是二叉树。
- 递归数据结构特别适合采用递归法求解。

【例2.7】 给定一个不带头结点的单链表 h ，设计一个递归算法删除其中所有结点值为 x 的结点。

解

设大问题 $f(h, x)$ 的功能是删除以 h 为首结点的单链表中所有结点值为 x 的结点。

$f(h, x) \equiv$ 不做任何事件

当 $h=NULL$ 时

$f(h, x) \equiv$ 删除 h 结点；

让 h 指向原后继结点； $f(h, x)$

当 $h \rightarrow val = x$ 时

$f(h, x) \equiv f(h \rightarrow next, x)$

当 $h \rightarrow val \neq x$

```

void Delallx(ListNode *&h,int x) {
//删除L中所有结点值为x的结点
    ListNode *p;
    if (h==NULL) return;
    if (h->val==x) {
        p=h; h=h->next;
        delete p;
        Delallx(h,x);
    }
    else Delallx(h->next,x);
}

```

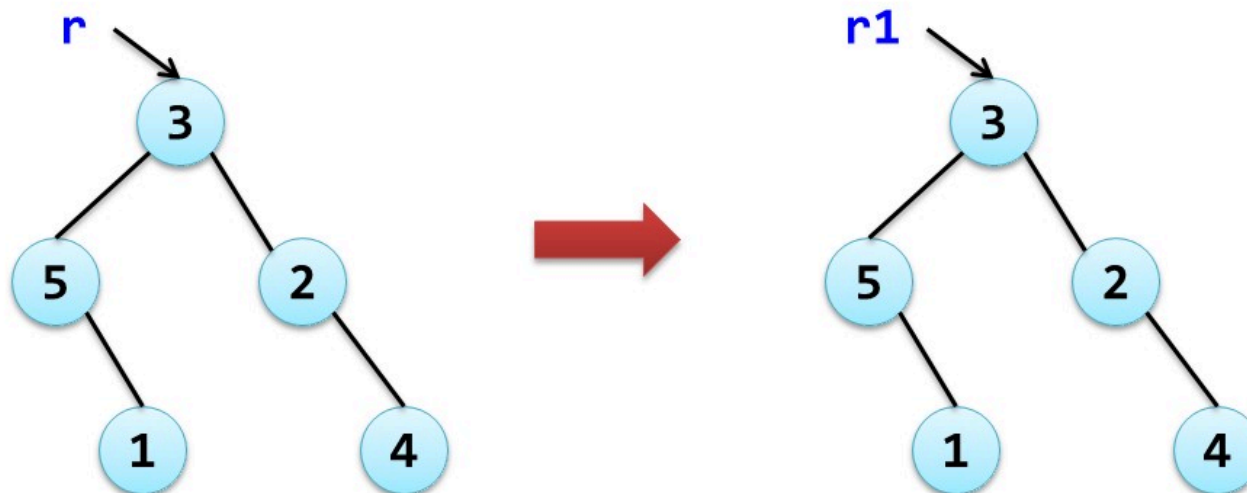
//删除结点值为x的结点
//此时减少了一个结点



【例2.8】克隆二叉树（LintCode375★★）。给定一棵二叉树，设计一个递归算法返回一个它的深度拷贝。

要求设计如下成员函数：

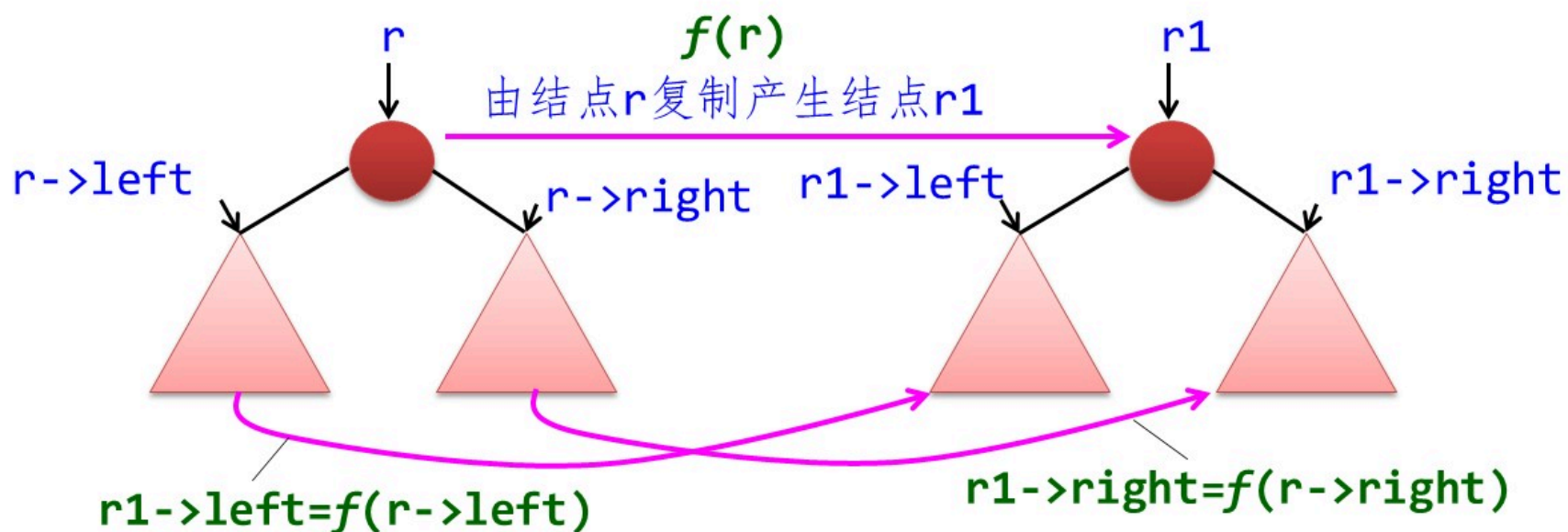
```
TreeNode * cloneTree(TreeNode* r) { }
```



47/91

解

设大问题 $f(r)$ 的功能是由二叉树 r 复制产生另一棵二叉树 $r1$ 并返回 $r1$ 。



```
class Solution {
public:
    TreeNode* cloneTree(TreeNode* r) {
        if (r==NULL) return NULL;
        else {
            TreeNode* r1=new TreeNode(r->val);
            r1->left=cloneTree(r->left);
            r1->right=cloneTree(r->right);
            return r1;
        }
    }
};
```

EXE 执行用时为**41ms**，内存消耗为**5.37MB**。

49/91

2.2.4 基于归纳思想的递归算法设计

- 基于归纳思想的递归算法设计通常不像基于递归数据结构的递归算法设计那样直观，需要通过对求解问题的深入分析，提炼出求解过程中的相似性而不是数据结构的相似性。
- 例如设大问题为 $f(n)$ ，小问题为 $f(n-1)$ 或者 $f(n/2)$ 等。
- 然后归纳出大小问题之间隐含的递推关系。

输入格式：输入数据由多行组成，每行包含一个整数 n ，表示该测试实例的长方形方格的规格是 $2 \times n$ ($0 < n \leq 50$)。

输出格式：对于每个测试实例，请输出铺放方案的总数，每个实例的输出占一行。

输入样例：

1

3

2

输出样例：

1

3

2

解

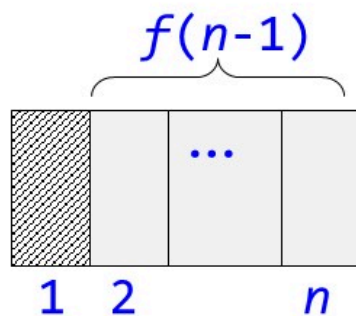
设 $f(n)$ 表示用 1×2 的骨牌铺满 $2 \times n$ 的一个长方形方格的铺放方案总数。

当 $n=1$ 时，用一块 1×2 的骨牌铺满，即 $f(1)=1$ 。

当 $n=2$ 时，用两块 1×2 的骨牌横向或者纵向铺满，即 $f(2)=2$ 。

当 $n > 2$ 时， $2 \times n$ 的一个长方形方格看成是高度为2的 n 个方格组成，编号依次是 $1 \sim n$ ，铺放分为如下情况：

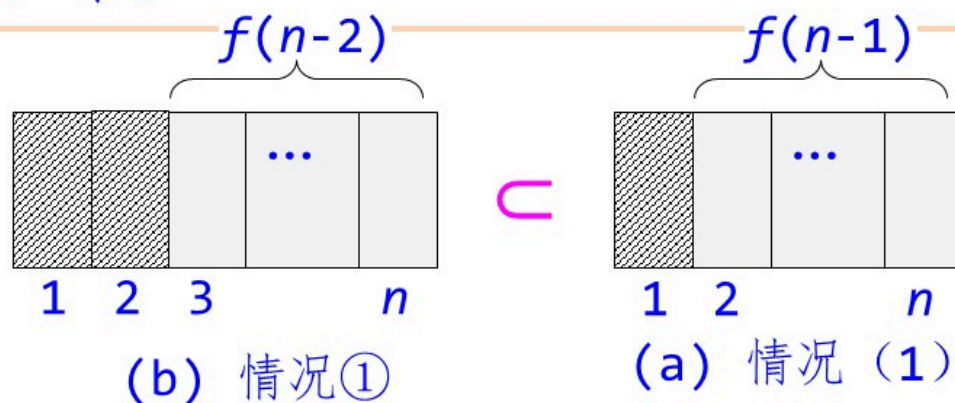
(1) 先铺好方格1，剩下 $2 \sim n$ 共 $n-1$ 个方格有 $f(n-1)$ 种铺放方案，如图 (a)所示，采用乘法原理，情况1的铺放方案总数 $= 1 * f(n-1) = f(n-1)$ 。



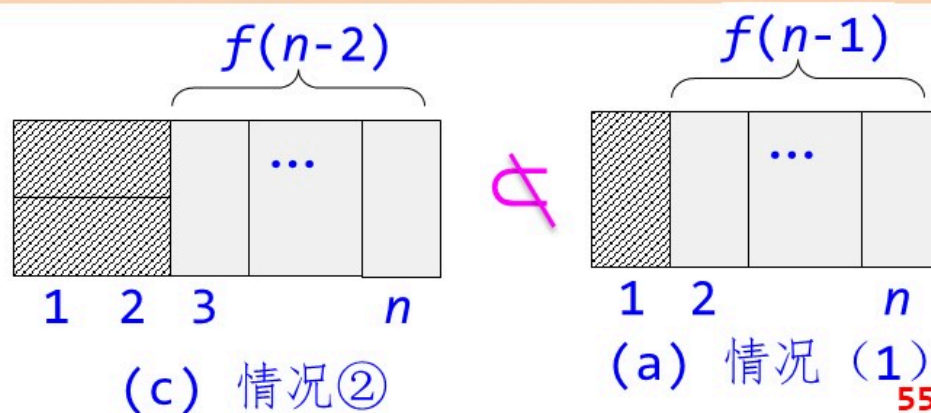
(a) 情况 (1)

(2) 先铺好方格**1**和方格**2**，剩下**3~n**共 **$n-2$** 个方格有 **$f(n-2)$** 种铺放方案。前面两个方格对应两种铺放方案：

①一种见图**(b)**，铺放方案总数= **$1*f(n-2)=f(n-2)$** ，但该铺放方案包含在情况**(1)**中。



②另一种见图**(c)**，铺放方案总数= **$1*f(n-2)=f(n-2)$** ，该铺放方案没有包含在情况**(1)**中。



55/91

采用加法原理，铺放方案总数 $f(n)=f(n-1)+f(n-2)$ 。



$$f(1)=1$$

$$f(2)=2$$

$$f(n)=f(n-1)+f(n-2)$$

当 $n>2$

如果直接采用递归算法求解会超时，可以采用如下迭代算法求解！

```

#include<iostream>
using namespace std;
long long Count(int n) {
    long long a=1,b=2,c;
    if(n==1)
        return a;
    else if(n==2)
        return b;
    else {
        for(int i=3;i<=n;i++) {
            c=a+b;
            a=b;
            b=c;
        }
        return c;
    }
}

```

//求铺放方案的总数
//a,b,c分别对应f(n-2),f(n-1),f(n)

$f(n)=f(n-1)+f(n-2)$

57/91

```
int main () {  
    int n;  
    while (~scanf("%d", &n))  
        printf ("%lld\n", Count(n));  
    return 0;  
}
```

EXE 执行用时为**31ms**，内存消耗为**1732KB**。



如何进一步改进时间性能？

58/91

2.3 直接插入排序

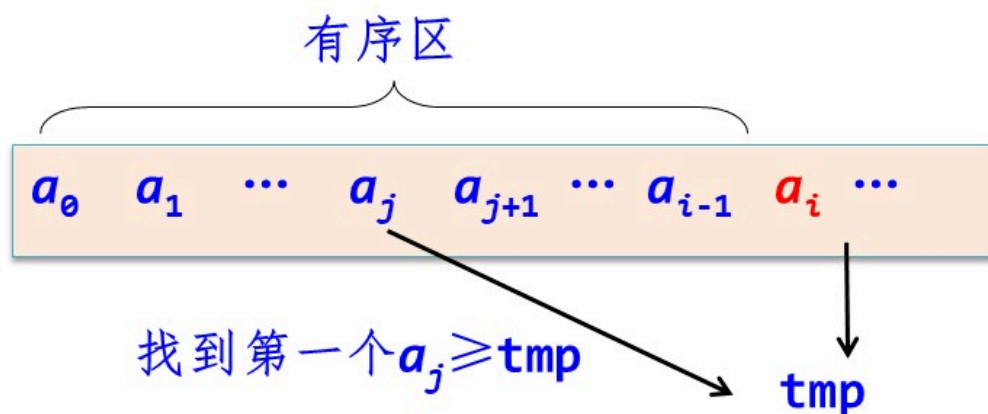
 **问题描述：**对于给定的含有 n 个整数元素的数组 a ，采用直接插入排序法对其按元素递增排序，设计相应的递归排序算法。

解

设大问题 $f(a, n, i)$ 用于实现 $a[0..i]$ （共 $i+1$ 个元素）的递增排序，则小问题 $f(a, n, i-1)$ 用于实现 $a[0..i-1]$ （共 i 个元素）的递增排序。大问题 $f(a, n, i)$ 的操作如下：

（1）当 $i=0$ 时，排序区间为 $a[0..0]$ ，其中只有一个元素 $a[0]$ ，显然是有序的，不需要做任何操作。

(2) 当 $i > 0$ 时, 先调用 $f(a, n, i-1)$ 实现 $a[0..i-1]$ 的递增排序, 然后将 a_i 有序插入到有序区 $a[0..i-1]$ 中。



$f(a, n, i) \equiv$ 不做任何事情, 算法结束

$f(a, n, i) \equiv f(a, n, i-1);$

将 $a[i]$ 有序插入到 $a[0..i-1]$ 中

当 $i=1$ 时

其他情况

```

void insertsort(int a[],int n,int i) {    //递归算法
    if(i>0) {
        insertsort(a,n,i-1);
        if(a[i]<a[i-1]) {                //反序时
            int tmp=a[i];
            int j=i-1;
            do {                          //找a[i]的插入位置
                a[j+1]=a[j];              //将大于a[i]的元素后移
                j--;
            } while(j>=0 && a[j]>tmp);
            a[j+1]=tmp;                   //在j+1处插入a[i]
        }
    }
}

```

61/91

 **insertsort** 算法分析：设 $f(a, n, n-1)$ 的执行时间为 $T(n)$ ，则设 $f(a, n, n-2)$ 的执行时间为 $T(n-1)$ ：

$$T(1)=O(1)$$

$$T(n)=T(n-1)+O(n)$$

当 $n>1$ 时

可以推出 $T(n)=O(n^2)$ 。

2.4 0/1背包问题

 **问题描述：**有 n 个物品，物品编号为 $0 \sim n-1$ ，重量为 $\{w_0, w_1, \dots, w_{n-1}\}$ ，价值为 $\{v_0, v_1, \dots, v_{n-1}\}$ ，给定一个容量为 W 的背包。现在从这些物品中选取若干物品装入该背包的方案，每个物品要么选中要么不选中，要求选中的物品总重量不超过背包容量并且具有最大的价值。

物品编号	重量	价值
0	5	4
1	3	4
2	2	3
3	1	1

$W=6$

63/91

解

- 用数组 x 存放一个装入方案, $x[i]=1$ 表示选择物品 i , $x[i]=0$ 表示不选择物品 i (x 称为解向量)。
- 设大问题 $f(w, v, i, rw)$ 表示考虑物品 $i \sim n-1$ (共 $n-i$ 个) 并且背包剩余容量为 rw 时的最大价值。
- 小问题 $f(w, v, i+1, rw)$ 表示考虑物品 $i+1 \sim n-1$ (共 $n-i-1$ 个) 并且此时背包剩余容量为 rw 时的最大价值。
- 大小问题需要考虑的物品个数相差一个即物品 i 。

考虑物品 $i \sim n-1$, 剩余容量为 rw 的最大价值



求解大问题 $f(w, v, i, rw)$ 的递归模型:

$$f(w, v, i, rw) = 0$$

$i = n$ 或 $rw < 0$

$$f(w, v, i, rw) = \max \{ \begin{array}{l} f(w, v, i+1, rw - w[i]) + v[i], \\ f(w, v, i+1, rw) \end{array} \}$$

其他情况

不选择物品 i

选择物品 i

当调用 $f(w, v, 0, W)$ 时便求出物品重量和价值分别为 w 和 v 、背包容量为 W 的最大价值。

$f(w, v, i, rw) = 0$	$i = n$ 或 $rw < 0$
$f(w, v, i, rw) = \max\{$	其他情况
$\quad f(w, v, i+1, rw - w[i]) + v[i],$	
$\quad f(w, v, i+1, rw) \}$	

```

vector<int> x;                                //解向量
int dfs(vector<int>&w, vector<int>&v, int i, int rw) {
    int n = w.size();
    if (i >= n || rw < 0) return 0;           //递归出口
    int maxv1 = 0;
    if (rw >= w[i]) {
        maxv1 = dfs(w, v, i+1, rw - w[i]) + v[i]; //选择物品i
    }
    int maxv2 = dfs(w, v, i+1, rw);           //不选择物品i
}

```

```

    if(maxv1>maxv2) {
        x[i]=1;
        return maxv1;
    }
    else {
        x[i]=0;
        return maxv2;
    }
}

```

$f(w, v, i, rw) = 0$

$i = n$ 或 $rw < 0$

$f(w, v, i, rw) = \max \{ \quad f(w, v, i+1, rw - w[i]) + v[i], \quad$
 $\quad f(w, v, i+1, rw) \}$

其他情况

```

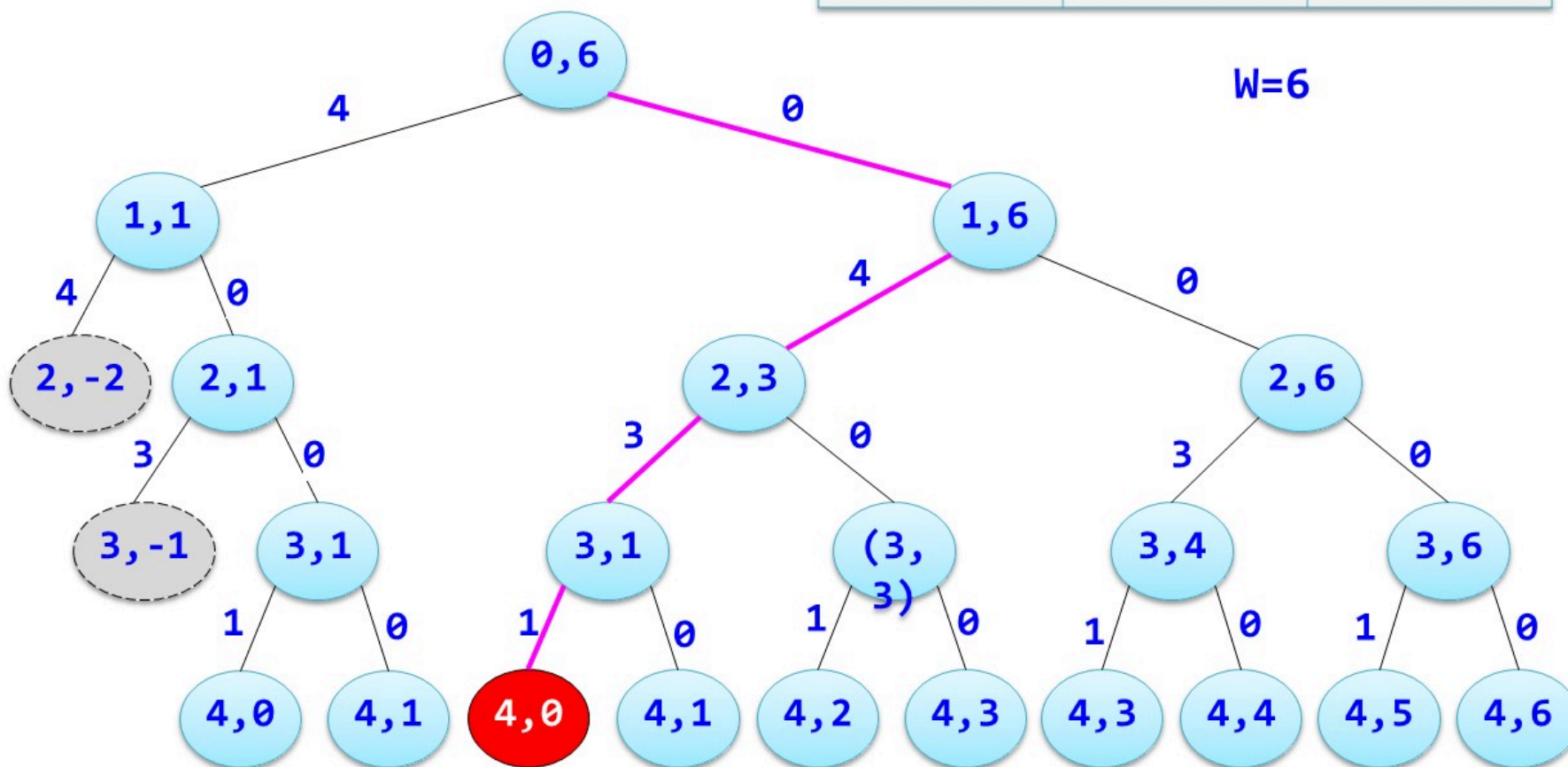
void Knap(vector<int>&w,vector<int>&v,int W) {
    int n=w.size();
    x.resize(n);
    int ans=dfs(w,v,0,W);

    printf("最佳方案为:\n");
    for(int i=0;i<n;i++)
        if(x[i]==1) printf(" 选中物品%d[%d,%d]\n",
                                i,w[i],v[i]);
    printf("最优总价值:%d\n",ans);
}

```

用 (i, rw) 表示状态

物品编号	重量	价值
0	5	4
1	3	4
2	2	3
3	1	1



答案：选择物品1、物品2和物品3，最优总价值：8

69/91



【例2.11】背包问题（LintCode92★★）。在 n 个物品中挑选若干物品装入背包，最多能装多满？假设背包的大小为 m ，每个物品的大小为 A_i （每个物品只能选择一次），不可以将物品进行切割。

例如， $A=\{3, 4, 8, 5\}$ ， $m=10$ ，答案是9。

要求设计如下成员函数：

```
int backPack(int m,vector<int> &A) { }
```

解

- 与0/1背包问题类似。
- 只是将0/1背包问题中求最大价值改为求满足容量限制的最大总重量（大小）。

71/91

```

class Solution {
public:
    int backPack(int m,vector<int> &A) {           //求解算法
        dfs(A,0,m);
    }

    int dfs(vector<int> &A,int i,int rm) {        //递归算法
        int n=A.size();
        if(i>=n || rm<0) return 0;
        int maxw1=0;
        if(rm>=A[i]) maxw1=dfs(A,i+1,rm-A[i])+A[i];
        int maxw2=dfs(A,i+1,rm);
        return max(maxw1,maxw2);
    }
};

```

72/91



- 上述程序提交时**超时** (Time Limit Exceeded)。
- 用 $f(i, rm)$ 表示考虑物品 $i \sim n-1$ 并且背包剩余容量为 rm 时装入背包的最大物品总重量。
- 大问题 $f(0, m)$ 转换为若干小问题，超时的原因是其中存在大量重复计算的小问题。例如 $f(2, 10)$ 可能重复计算**10**次。
- 为了避免重复计算，设计一个二维数组 dp ，用 $dp[i][rm]$ 存放 $f(i, rm)$ 的结果，初始化 dp 所有元素为**-1**，一旦 $f(i, rm)$ 已经求出，置 $dp[i][rm]$ 为 $f(i, rm)$ 。
- 在求一个 $f(i, rm)$ 子问题时，先判断 $dp[i][rm]$ 是否等于**-1**，若不是说明该子问题已经求出，直接返回 $dp[i][rm]$ 即可。

```
class Solution {  
    vector<vector<int>> dp;  
    int n;  
public:  
    int backPack(int m,vector<int> &A) {        //求解算法  
        n=A.size();  
        dp=vector<vector<int>>(n+1,vector<int>(m+1,-1));  
        dfs(A,0,m);  
    }  
}
```

```

int dfs(vector<int> &A,int i,int rm) {           //递归算法
    if(dp[i][rm]!=-1) return dp[i][rm];
    if(i>=n || rm<0) {
        dp[i][rm]=0;
        return 0;
    }
    int maxw1=0;
    if(rm>=A[i]) maxw1=A[i]+dfs(A,i+1,rm-A[i]);
    int maxw2=dfs(A,i+1,rm);
    dp[i][rm]=max(maxw1,maxw2);
    return dp[i][rm];
}
};

```



执行用时为**605ms**，内存消耗为**35.76MB**。

75/91



2.5 求表达式值

 **问题描述：**基本计算器（LeetCode224★★★）。给定一个长度为 n （ $1 \leq n \leq 300000$ ）字符串表达式 s ， s 是仅由整数、 $'+'$ 、 $'-'$ 、 $'('$ 、 $'('$ 、 $'')$ 和空格组成的一个有效算术表达式，其中 $'+'$ 不能用作一元运算（例如， $" +1 "$ 和 $" +(2+3) "$ 是无效的），而 $'-'$ 可以用作一元运算（例如， $" -1 "$ 和 $" -(2+3) "$ 是有效的），所有整数和计算结果均可以用`int`表示。实现一个基本计算器来计算并返回字符串 s 的值。

例如， $s = "(1+(4+5+2)-3)+(6+8)"$ ，返回表达式值是23。

要求设计如下成员函数：

```
int calculate(string s) { }
```

解

- 用整数变量`ans`存放表达式`s`的值（初始为0）。
- 表达式`s`中仅包含 '+' 和 '-' 运算符，它们的优先级相同，如果`s`中没有括号，可以将`s`分割为 `sign d` 序列，其中 `sign` 为整数 `d` 的符号（`sign=1` 表示 '+', `sign=-1` 表示 '-'）。
- 例如 `s="1-5+8"` 可以分割为 `+1`, `-5`, `+8`，这样表达式值 $= (+1) + (-5) + (+8) = 4$ 。也就是说将所有分割的整数相加得到最后结果 `ans`。

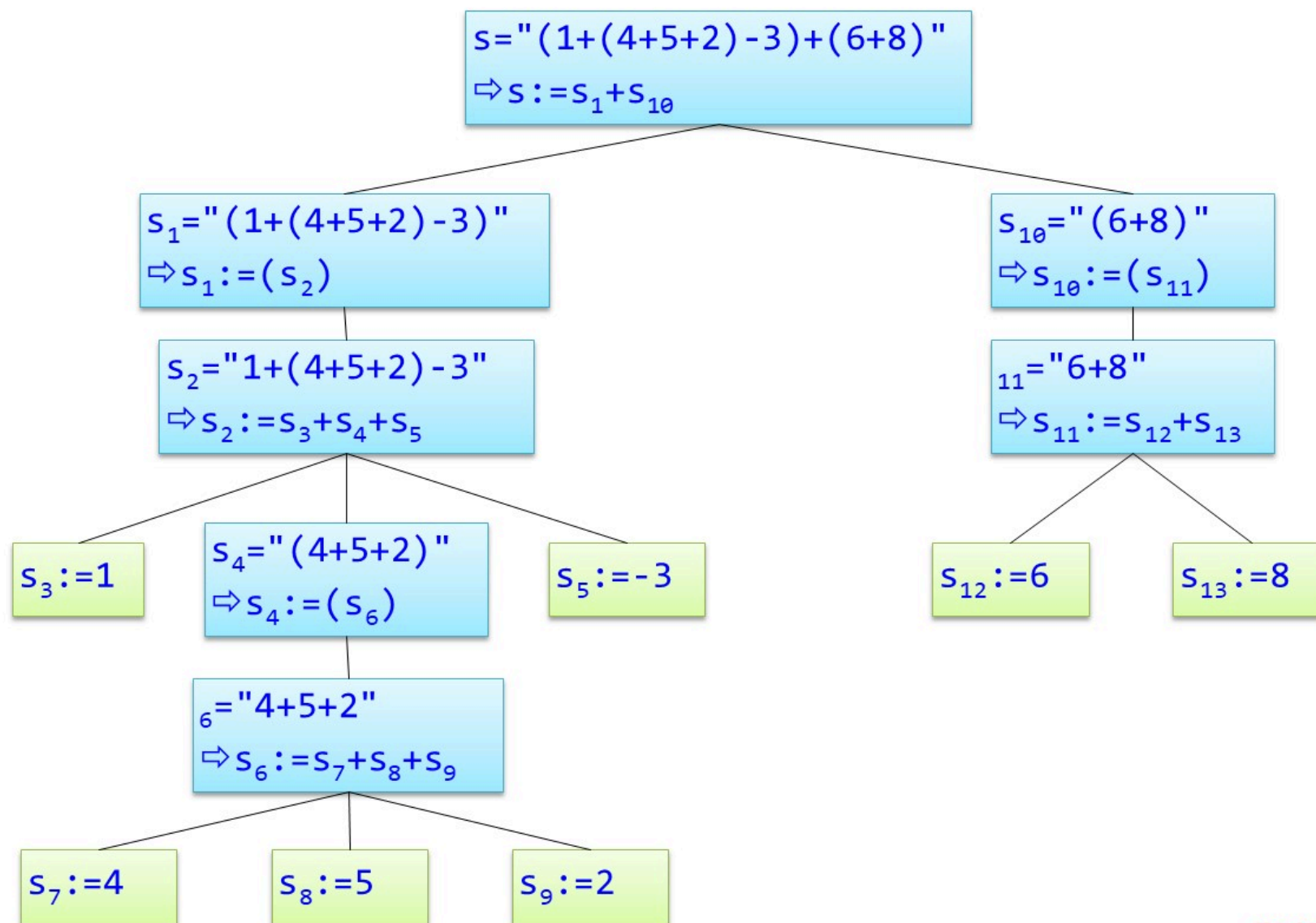
- 如果 **s** 中存在括号，任何一个匹配的括号对中也是一个有效算术表达式。
- 可以采用 **BNF** 范式表示有效算术表达式 **s** 如下（其中 **d** 是整数串）：

s := **d**

s := **s** + **s** | **s** - **s**

s := (**s**)

$s = "(1+(4+5+2)-3)+(6+8)"$ 采用BNF范式表示



79/91

这样用 **i** 遍历 **s**，**ans=0**，用 **sign** 表示当前整数或者子表达式值的符号（默认取 **sign=1**），求解过程如下：

- ① 遇到空格时直接跳过该空格。
- ② 遇到 '+' 时，置 **sign=1**，跳过该 '+'。
- ③ 遇到 '-' 时，置 **sign=-1**，跳过该 '-'。
- ④ 遇到数字字符时，将连续的数字字符转换为整数 **d**，其符号为 **sign**，则将 **sign*d** 累加到 **ans** 中。
- ⑤ 遇到 '('，后面开始一个子表达式（该子表达式以后面的第一个未处理的 ')' 结束），递归计算出该子表达式值 **sd**，其符号为 **sign**，则将 **sign*sd** 累加到 **ans** 中。
- ⑥ 遇到 ')'，表示表达式/子表达式处理结束，返回其值 **ans**。

例如，求 $s = "(1+(4+5+2)-3)+(6+8)"$ 表达式值的过程。

$$\begin{array}{c} s = "(1+(4+5+2)-3)+(6+8)" \\ \underbrace{\hspace{1.5cm}} \quad \underbrace{\hspace{1.5cm}} \\ \textcircled{1}11 \quad \textcircled{3}14 \\ \underbrace{\hspace{2.5cm}} \\ \textcircled{2}9 \\ \underbrace{\hspace{3.5cm}} \\ \textcircled{4}23 \end{array}$$

```
class Solution {  
public:  
    int calculate(string s) {                //求解算法  
        int i=0;  
        return dfs(s,i);  
    }  
}
```

```
int dfs(string s,int& i) {           //递归算法
    int n=s.size();
    int ans=0;
    int sign=1;                       //默认为 '+'
    while(i<n) {
        if(s[i]==' ') {              //遇到空格
            i++;                     //跳过空格
        }
        else if(s[i]=='+') {         //遇到 '+'
            sign=1;                  //指定为 '+'
            i++;                     //跳过 '+'
        }
    }
}
```

```

else if(s[i]=='-') {           //遇到 '-'
    sign=-1;                  //指定为 '-'
    i++;                      //跳过 '-'
}
else if(isdigit(s[i])) {      //遇到数字字符
    int d=0;
    while(i<n && isdigit(s[i])) { //将连续数字字符->整数d
        d=d*10+(s[i++]-'0');
    }
    ans+=d*sign;              //转换为带符号整数累加到ans
}
else if(s[i]=='(') {         //遇到 '('
    i++;                      //跳过 '('
    int sd=dfs(s,i);          //递归计算出子表达式值sd
    ans+=sd*sign;             //将sd累加到ans
}

```

84/91

```
        else {                                //遇到')'  
            i++;                             //跳过')'  
            return ans;                     //返回表达式/子表达式的值  
        }  
    }  
    return ans;  
}  
};
```



执行用时为**20ms**，内存消耗为**277.3MB**。



【例2.12】字符串解码（LeetCode394★★）。问题描述见第1章例1.9，这里采用递归方法求解。

- 自学
- 体会两种解法之间的差别

2.6 计算递推式

2.6.1 直接展开法

- 求解递推式最自然的方法是将其反复展开。
- 即直接从递归式出发，一层一层地往前递推，直到最前面的初始条件为止，就得到了问题的解。
- 例如，前面 $\text{Hanoi}(n, x, y, z)$ 递归算法的时间复杂度计算就是直接展开法。

【例2.13】 分析以下递推式的时间复杂度。

$$T(0)=0$$

$$T(n)=nT(n-1)+n!$$

当 $n \geq 1$

解

$$T(0)=0$$

$$T(n)=nT(n-1)+n! \quad \text{当 } n \geq 1$$

先展开一次即

$$\begin{aligned} T(n) &= nT(n-1) + n! = n[(n-1)T(n-2) + (n-1)!] + n! \\ &= n(n-1)T(n-2) + 2n!。 \end{aligned}$$

由于 $n! = n(n-1)(n-2)!$ ，则 $n(n-1) = \frac{n!}{(n-2)!}$

$$\text{所以有 } T(n) = n(n-1)T(n-2) + 2n! = \frac{n!T(n-2)}{(n-2)!} + 2n! = n! \left(\frac{T(n-2)}{(n-2)!} + 2 \right)$$

构造一个辅助函数 $g(n) = \frac{T(n-2)}{(n-2)!} + 2$ ，令 $T(n) = n!g(n)$ ，可以推出 $T(0) = g(0) = 0$ 。

$$\text{这样 } T(n-1) = (n-1)!g(n-1)$$

代入递推式 $T(n) = nT(n-1) + n!$ 有 $n!g(n) = n(n-1)!g(n-1) + n!$ 。

两边除 $n!$ 得到 $g(n) = g(n-1) + 1$ 。

采用展开法得到 $g(n) = n$ 。因此：

$$T(n) = n!g(n) = n \times n! = O(n \times n!)。$$

89/91

3.6.2 递归树方法

- 用递归树求解递推式的基本过程是，展开递推式，构造对应的递归树，然后把每一层的时间进行求和，从而得到算法执行时间的估计。
- 再用时间复杂度形式表示。

【例2.14】分析以下递推式的时间复杂度。

$$T(n)=1$$

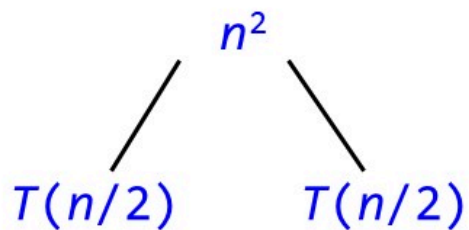
当 $n=1$

$$T(n)=2T(n/2)+n^2$$

当 $n>1$

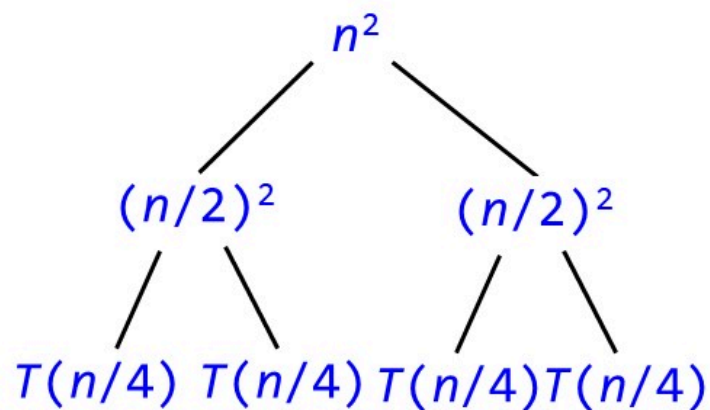
解

$T(n)$

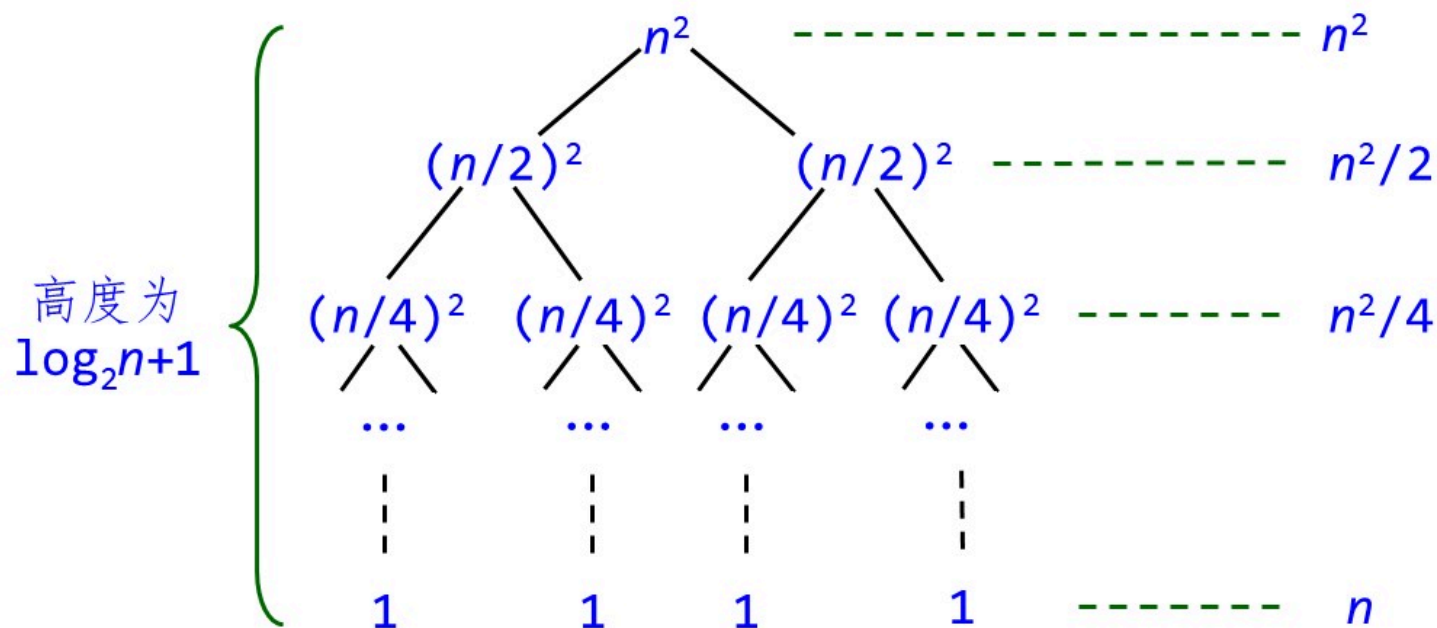


(a) 初始

(b) 展开 $T(n)$



(c) 展开 $T(n/2)$



- 第1层的问题规模为 n ，第2的层的问题规模为 $n/2$ ，依此类推，当展开到第 $k+1$ 层，其规模为 $n/2^k=1$ ，所以递归树的高度为 $\log_2 n + 1$ 。
- 所有层： $T(n) = n^2 + n^2/2 + \dots + n^2/2^{k-1} + \dots + n = O(n^2)$ 。

2.6.3 主方法

求解如下形式递推式的一般方法

$$T(1)=c$$

$$T(n)=aT(n/b)+f(n)$$

当 $n>1$ 时

- 其中 $a \geq 1$, $b > 1$ 为常数, n 为非负整数。
- $T(n)$ 表示算法的执行时间。
- 该算法将规模为 n 的原问题分解成 a 个子问题, 每个子问题的大小为 n/b , $f(n)$ 表示分解原问题和合并子问题解得到答案的时间。

93/91

主定理： $T(n)$ 的计算如下：

- ① 若对于某个常数 $\varepsilon > 0$ ，有 $f(n) = O(n^{\log_b a - \varepsilon})$ ，称为 $f(n)$ 多项式地小于 $n^{\log_b a}$ ，即 $f(n)$ 与 $n^{\log_b a}$ 的比值小于等于 $n^{-\varepsilon}$ ，则 $T(n) = \Theta(n^{\log_b a})$ 。
- ② 若 $f(n) = \Theta(n^{\log_b a})$ ，即 $f(n)$ 多项式的等于 $n^{\log_b a}$ ，则 $T(n) = \Theta(n^{\log_b a} \log_2 n)$ 。
- ③ 若对于某个常数 $\varepsilon > 0$ ，有 $f(n) = \Omega(n^{\log_b a + \varepsilon})$ ，称为 $f(n)$ 多项式地大于 $n^{\log_b a}$ ，即 $f(n)$ 与 $n^{\log_b a}$ 的比值大于等于 n^ε ，并且满足 $af(n/b) \leq cf(n)$ ，其中 $c < 1$ ，则 $T(n) = \Theta(f(n))$ 。

【例2.16】采用主定理求以下递推式的时间复杂度。

$$T(n)=1$$

当 $n=1$

$$T(n)=4T(n/2)+n$$

当 $n>1$

解

$a=4, b=2, n^{\log_b a} = n^2, f(n)=n=O(n^{\log_b a - \varepsilon})$, 这里 $\varepsilon=1$,
即 $f(n)$ 多项式地小于 $n^{\log_b a}$, 满足情况①, 所以 $T(n)=$
 $= \Theta(n^{\log_b a}) = \Theta(n^2)$ 。

【例2.17】采用主定理求以下递推式的时间复杂度。

$$T(n)=1 \quad \text{当 } n=1$$

$$T(n)=3T(n/4)+n\log_2 n \quad \text{当 } n>1$$

解

- $a=3, b=4, f(n)=n\log_2 n$ 。 $n^{\log_b a} = O(n^{0.793})$ ，显然 $f(n)$ 的阶大于 $n^{0.793}$ （因为 $f(n)=n\log_2 n > n^1 > n^{0.793}$ ）。
- 对于足够大的 n ，
$$af(n/b)=3(n/4)\log_2(n/4)=(3/4)n\log_2 n - 3n/2$$
$$\leq (3/4)n\log_2 n = cf(n), \text{ 这里 } c=3/4, \text{ 满足正规性条件}$$
- 按照主定理中情况③，有 $T(n)=\Theta(f(n))=\Theta(n\log_2 n)$ 。

96/91

可以这样简化主定理，如果递推式如下：

$$T(1)=c$$

$$T(n)=aT(n/b)+cn^k$$

当 $n>1$ 时

其中 a 、 b 、 c 、 k 都是常量，则：

- ① 若 $a > b^k$ ，则 $T(n) = \Theta(n^{\log_b a})$ 。
- ② 若 $a = b^k$ ，则 $T(n) = \Theta(n^k \log_b n)$ 。
- ③ 若 $a < b^k$ ，则 $T(n) = \Theta(n^k)$ 。



思政

98/91